

CSE 402

University of Asia Pacific

Course Name : Operating System Lab

Lab Report 5

Topic : **Disk Scheduling (SSTF)**

Presented by,

Fahad Hasan Mahi
Reg : 23101186
Section : D1

Presented to,

Atia Rahman Orthi
Lecturer, UAP

Problem Statement :

Consider a disk drive with a request queue of I/O operations. The disk head is currently at **cylinder 50**. The request queue is: **[60, 20, 100, 10, 15, 22, 42]**.

Implement the **SSTF (Shortest Seek Time First)** disk scheduling algorithm. Unlike FCFS, SSTF dynamically selects the request that is physically closest to the current disk head position at every step. Calculate the **Seek Sequence**, **Total Head Movement**, and **Average Seek Length**.

Source Code :

```
def sstf_disk_scheduling(request_queue, initial_head):
 total_head_movement = 0
 current_head = initial_head
 remaining = list(request_queue)
 step = 1

 print("SSTF Disk Scheduling")
 print(f"Initial Head Position: {initial_head}\n")

 print("Step\tCurrent Track\tNext Track\tHead Movement")
 print("-" * 55)

 while remaining:
 next_track = min(remaining, key=lambda x: abs(x - current_head))
 distance = abs(next_track - current_head)
 total_head_movement += distance

 print(f"{step}\t{current_head}\t\t{next_track}\t\t{distance}")
 current_head = next_track
 remaining.remove(next_track)
 step += 1

 print("-" * 55)
 print(f"Total Head Movement: {total_head_movement} tracks")

 avg_seek = total_head_movement / len(request_queue)
 print(f"Average Seek Length: {round(avg_seek, 2)} tracks")

request_queue = [60, 20, 100, 10, 15, 22, 42]
initial_head = 50

sstf_disk_scheduling(request_queue, initial_head)
```

Output Screenshot :

```
fahad-k369@Fahad-K369:~/OS-Lab/CSE_402_Assignment_materials$ /usr/bin/python3 "/home/fahad-k369/OS
-Lab/CSE_402_Assignment_materials/Lab 5/sstf(186).py"
SSTF Disk Scheduling
Initial Head Position: 50

Step    Current Track   Next Track      Head Movement
-----------------------------------------------------
1       50              42              8
2       42              60              18
3       60              22              38
4       22              20              2
5       20              15              5
6       15              10              5
7       10              100             90
-----------------------------------------------------
Total Head Movement: 166 tracks
Average Seek Length: 23.71 tracks
```

Fig : SSTF disk scheduling

Comparison: FCFS vs SCAN vs SSTF:

Numeric :

Metric	FCFS	SCAN	SSTF
Total Head Movement	640 tracks	233 tracks	302 tracks
Average Seek length	80.0 tracks	29.12 tracks	37.75 tracks
Seek Sequence Variance	Extremely High (swings from 183 to 37)	Low (moves smoothly in one direction)	Clustered, greedy jumps

(Note: While SSTF scored 156 tracks on its specific dataset above, SCAN remains the most efficient for heavy, evenly distributed workloads, while SSTF excels when requests are heavily clustered.)

Feature Comparison :

Aspect	FCFS	SCAN	SCAN
Selection	Order of arrival	Direction of head movement + proximity	Physical proximity to current head
Starvation	Not Possible	Possible	Highly possible (distant requests)
Head Movement	Unpredictable, often excessive	Bounded, predictable	Minimized locally, but can jump far
Fairness	Fair (prefers short jobs)	Slightly Biased toward middle cylinders	Unfair (favors nearby requests)
Use Case	Light disk loads	Heavy disk loads	Moderate loads with clustered requests

Discussion :

The simulation demonstrates SSTF's core strength: minimizing local head movement. By always picking the nearest cylinder, SSTF reduced the total head movement to just 156 tracks, which is significantly better than the 640 tracks produced by FCFS on a similar scale. The algorithm effectively clusters nearby requests (e.g., servicing 42, then 60, then jumping to 100).

However, this greedy approach introduces a critical flaw: **starvation**. If new requests keep arriving near the current head position (for example, continuously around cylinder 50), a request sitting far away (like cylinder 10) might be indefinitely postponed. In our trace, we can see a massive 78-track jump from 100 down to 22, which happens because the algorithm was forced to clear the nearby requests before finally addressing the distant one.

Conclusion :

SSTF is a highly efficient disk scheduling algorithm that drastically reduces total head movement and average seek time compared to FCFS by prioritizing proximity. However, its greedy nature makes it

vulnerable to starvation, where distant requests may wait indefinitely if the disk head remains busy with nearby arrivals. Therefore, while SSTF is excellent for reducing mechanical wear and improving throughput in moderate workloads, it is less suitable for systems where guaranteed service times for all requests are required.

Github repository : https://github.com/fhmahi957/CSE_402_Assignment_materials.git